

Inspect Rich Documents with Gemini Multimodality and Multimodal RAG Challenge Lab

Note: if you encounter an authentication error when running the cells in the notebook, go to **Agent Platform > Dashboard**, and click on **Enable All Recommended APIs**. Then, re-run the failed cell, and continue the lab.

Setup and requirements

Install Gen AI SDK for Python and other dependencies

Run the following four cells below before you get to Task 1. Be sure to add your current project ID to the cell titled **Define Google Cloud project information**.

```
In [ ]: # "RUN THIS CELL AS IS"

# Install required python packages and other dependencies
%pip install --upgrade --quiet google-genai
%pip install --quiet pymupdf
```

Restart current runtime

You must restart the runtime in order to use the newly installed packages in this Jupyter runtime. You can do this by running the cell below, which will restart the current kernel.

```
In [ ]: # "RUN THIS CELL AS IS"

import IPython

# Restart the kernel after libraries are loaded.

app = IPython.Application.instance()
app.kernel.do_shutdown(True)
```

⚠ **The kernel is going to restart. Please wait until it is finished before continuing to the next step.** ⚠

Define Google Cloud project information

```
In [ ]: # "RUN THIS CELL AS IS"

import os
PROJECT_ID = "qwiklabs-gcp-01-f261482ed36e"
LOCATION = "global"
```

Initialize Gen AI

Initialize the Gen AI SDK for Python for your project:

```
In [ ]: # "RUN THIS CELL AS IS"

# Initialize Gen AI.
from google import genai
client = genai.Client(enterprise=True, project=PROJECT_ID, location=LOCATION)
```

Task 1. Generating Multimodal Insights with Gemini

Gemini is multimodal model that supports multimodal prompts. You can include text, image(s), and video in your prompt requests and get text or code responses.

To complete Task 1, follow the instructions at the top of each notebook cell:

- Run the cells with the comment "RUN THIS CELL AS IS".
- Complete and run the cells with the comment "COMPLETE THE MISSING PART AND RUN THIS CELL".

Note: Ensure you can see the weather related data in the response that is printed.

Setup and requirements for Task 1

Import libraries

```
In [ ]: from google.genai import types
from google.genai.types import Part
from PIL import Image as PIL_Image
```

Load Gemini 3.5 Flash Model

```
In [ ]: # "RUN THIS CELL AS IS"

multimodal_model = "gemini-3.5-flash"
```

Define helper functions

In []: # "RUN THIS CELL AS IS"

```
import http.client
import io
import typing
import urllib.request

import IPython.display
from google.genai import types
from PIL import Image as PIL_Image
from PIL import ImageOps as PIL_ImageOps

def display_images(
    images: typing.Iterable[PIL_Image.Image],
    max_width: int = 600,
    max_height: int = 350,
) -> None:
    for image in images:
        pil_image = image
        if pil_image.mode != "RGB":
            # RGB is supported by all Jupyter environments (e.g. RGBA is not yet)
            pil_image = pil_image.convert("RGB")
        image_width, image_height = pil_image.size
        if max_width < image_width or max_height < image_height:
            # Resize to display a smaller notebook image
            pil_image = PIL_ImageOps.contain(pil_image, (max_width, max_height))
        IPython.display.display(pil_image)

def get_image_bytes_from_url(image_url: str) -> bytes:
    with urllib.request.urlopen(image_url) as response:
        response = typing.cast(http.client.HTTPResponse, response)
        image_bytes = response.read()
    return image_bytes

def load_image_from_url(image_url: str) -> PIL_Image.Image:
    image_bytes = get_image_bytes_from_url(image_url)
    return PIL_Image.open(io.BytesIO(image_bytes))

def display_content_as_image(content: str | PIL_Image.Image | types.Part) -> bool:
    if not isinstance(content, PIL_Image.Image):
        return False
    display_images([content])
    return True

def display_content_as_video(content: str | PIL_Image.Image | types.Part) -> bool:
    if not isinstance(content, types.Part):
        return False
    part = typing.cast(types.Part, content)
    file_path = part.file_data.file_uri.removeprefix("gs://")
    video_url = f"https://storage.googleapis.com/{file_path}"
```

```

IPython.display.display(IPython.display.Video(video_url, width=600))
return True

def print_multimodal_prompt(contents: list[str | PIL_Image.Image | types.Part]):
    """
    Given contents that would be sent to Gemini,
    output the full multimodal prompt for ease of readability.
    """
    for content in contents:
        if display_content_as_image(content):
            continue
        if display_content_as_video(content):
            continue
        print(content)

```

Task 1.1. Image understanding across multiple images

```

In [ ]: # "RUN THIS CELL AS IS"

# You're going to work with provided variables in this task.
# First, review and describe the content/purpose of each variable below.

image_ask_first_1_url = "https://storage.googleapis.com/spls/gsp520/Google_Branding
image_dont_do_this_1_url = "https://storage.googleapis.com/spls/gsp520/Google_Brand
image_ask_first_1 = load_image_from_url(image_ask_first_1_url)
image_dont_do_this_1 = load_image_from_url(image_dont_do_this_1_url)

instructions = "Instructions: Consider the following image that contains text:"
prompt1 = "What is the title of this image"
prompt2 = ""
Answer the question through these steps:
Step 1: Identify the title of each image by using the filename of each image.
Step 2: Describe the image.
Step 3: For each image, describe the actions that a user is expected to take.
Step 4: Extract the text from each image as a full sentence.
Step 5: Describe the sentiment for each image with an explanation.

Answer and describe the steps taken:
"""

```

Create an input for the multimodal model

```

In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Now, you're going to create an input for your multimodal model. Create your conte

contents = [instructions,
            prompt2,
            image_ask_first_1,
            image_dont_do_this_1]

```

Generate responses from the multimodal model

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the next part of this task, you're going to generate responses from the multimodal

responses = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=contents
)
```

Display the prompt and responses

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the last part of this task, you're going to print your contents and responses

# Hint: "\n" inserts a newline character for clearer separation between the sections

print("=== Prompts ===\n")
print_multimodal_prompt(contents)
print("\n=== Model Responses ===\n")
print(responses.text)
```

To verify your work for Task 1.1, click **Check my progress** in the lab instructions.

Task 1.2. Similarity/Differences between images

Explore the variables of the task

```
In [ ]: # "RUN THIS CELL AS IS"

# You're going to work with provided variables in this task. First, review and describe the variables.

image_ask_first_3_url = "https://storage.googleapis.com/spls/gsp520/Google_Branding"
image_dont_do_this_3_url = "https://storage.googleapis.com/spls/gsp520/Google_Branding"
image_ask_first_3 = load_image_from_url(image_ask_first_3_url)
image_dont_do_this_3 = load_image_from_url(image_dont_do_this_3_url)

prompt1 = """
Consider the following two images:
Image 1:
"""

prompt2 = """
Image 2:
"""

prompt3 = """
1. What is shown in Image 1 and Image 2?
2. What is similar between the two images?
3. What is difference between Image 1 and Image 2 in terms of the text ?
"""
```

Create an input for the multimodal model

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Now, you're going to create an input for your multimodal model. Create your contents

contents = [
    prompt1,
    image_ask_first_3,
    prompt2,
    image_dont_do_this_3,
    prompt3
]
```

Set configuration parameters

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Now, you're going to set configuration parameters that will influence how the multimodal model generates responses.
# Temperature: Controls randomness. Lower values mean more predictable results, higher values mean more creative.
# Top p / Top k: Affects how the model chooses words. Explore different values to see how they affect the output.
# Other parameters: Check the model's documentation for additional options you might want to use.

# Store your configuration parameters in a generation_config variable. This improves performance.

generation_config = GenerationConfig(
    temperature=0.4,
    top_p=1.0,
    top_k=40,
    max_output_tokens=1024
)

# HERE WE GET ERROR BUT WE KEEP MOVING
```

Generate responses from the multimodal model

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the next part of this task, you're going to generate responses from a multimodal model using the configuration parameters you set.

responses = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=contents
)
```

Display the prompt and responses

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the last part of this task, you're going to print your contents and responses
```

```
# Hint: "\n" inserts a newline character for clearer separation between the section

print("=== Prompts ===\n")
print_multimodal_prompt(contents)
print("\n=== Model Responses ===\n")
print(responses.text)
```

To verify your work for Task 1.2, click **Check my progress** in the lab instructions.

Task 1.3. Generate a video description

Explore the variables of the task

```
In [ ]: # "RUN THIS CELL AS IS"

# You're going to work with provided variables in this task.
# First, review and describe the content/purpose of each variable below.

prompt = """
What is the product shown in this video?
What specific product features are highlighted in the video?
Where was this video filmed? Which cities around the world could potentially serve
What is the sentiment of the video?
"""

video = Part.from_uri(
    file_uri="gs://spl/spls/gsp520/google-pixel-8-pro.mp4",
    mime_type="video/mp4",
)
```

Create an input for the multimodal model

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Now, you're going to create an input for your multimodal model. Create your contents

contents = [prompt, video]
```

Generate responses from the multimodal model

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the next part of this task, you're going to generate responses from a multimodal

responses = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=contents
)
```

Display the prompt and responses

Note: If you encounter any authentication error below cell run, go to the **Navigation menu**, click **Agent Platform > Dashboard**, then click **"Enable all Recommended APIs"** Now, go back to cell 16, and run cells 16, 17 and below cell.

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the last part of this task, you're going to print your contents and responses

# Hint: "\n" inserts a newline character for clearer separation between the section

print("=== Prompts ===\n")
print(prompt)
print("\n=== Model Responses ===\n")
print(responses.text)
```

Proceed to Task 1.4 below (no progress check for Task 1.3 in lab instructions).

Task 1.4. Extract tags of objects throughout the video

Explore the variables of the task

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# You're going to work with provided variables in this task. First, review and desc

prompt = """
Answer the following questions using the video only:

Which particular sport is highlighted in the video?
What values or beliefs does the advertisement communicate about the brand?
What emotions or feelings does the advertisement evoke in the audience?
Which tags associated with objects featured throughout the video could be extracted
"""

video = Part.from_uri(
    file_uri="gs://spl/spls/gsp520/google-pixel-8-pro.mp4",
    mime_type="video/mp4",
)
```

Create an input for the multimodal model

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Now, you're going to create an input for your multimodal model. Create your conte

contents = [prompt, video]
```

Generate responses from the multimodal model

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the next part of this task, you're going to generate responses from a multimod

responses = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=contents
)
```

Display the prompt and responses

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the last part of this task, you're going to print your contents and responses

# Hint: "\n" inserts a newline character for clearer separation between the section

print("=== Prompts ===\n")
print(prompt)
print("\n=== Model Responses ===\n")
print(responses.text)
```

Proceed to Task 1.5 below (no progress check for Task 1.4 in lab instructions).

Task 1.5. Ask more questions about a video

Note: Although this video contains audio, Gemini does not currently support audio input and will only answer based on the video.

Explore the variables of the task

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# You're going to work with provided variables in this task.
# First, review and describe the content/purpose of each variable below.

prompt = """
Answer the following questions using the video only:

How does the advertisement portray the use of technology, specifically AI, in captu
What visual cues or storytelling elements contribute to the nostalgic atmosphere of
How does the advertisement depict the role of friendship and social connections in
Are there any specific features or functionalities of the phone highlighted in the

Provide the answer JSON.
"""

video = Part.from_uri(
    file_uri="gs://spls/gsp520/google-pixel-8-pro.mp4",
    mime_type="video/mp4",
)
```

Create an input for the multimodal model

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Now, you're going to create an input for your multimodal model. Create your contents
contents = [prompt, video]
```

Generate responses from the multimodal model

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the next part of this task, you're going to generate responses from a multimodal model

responses = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=contents
)
```

Display the prompt and responses

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the last part of this task, you're going to print your contents and responses

# Hint: "\n" inserts a newline character for clearer separation between the sections

print("=== Prompts ===\n")
print(prompt)
print("\n=== Model Responses (JSON) ===\n")
print(responses.text)
```

Proceed to Task 1.6 below (no progress check for Task 1.5 in lab instructions).

Task 1.6. Retrieve extra information beyond the video

Explore the variables of the task

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# You're going to work with provided variables in this task.
# First, review and describe the content/purpose of each variable below.

prompt = """
Answer the following questions using the video only:

How does the advertisement appeal to its target audience through its messaging and
What overall message or takeaway does the advertisement convey about the brand and
Are there any symbolic elements or motifs used throughout the advertisement to reinforce the message?
What is the best hashtag for this video based on the description ?
```

```

"""
video = Part.from_uri(
    file_uri="gs://spls/gsp520/google-pixel-8-pro.mp4",
    mime_type="video/mp4",
)

```

Create an input for the multimodal model

```

In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Now, you're going to create an input for your multimodal model. Create your contents
contents = [prompt, video]

```

Generate responses from the multimodal model

```

In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the next part of this task, you're going to generate responses from a multimodal model

responses = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=contents
)

```

Display the prompt and responses

```

In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# In the last part of this task, you're going to print your contents and responses

# Hint: "\n" inserts a newline character for clearer separation between the sections

print("=== Prompts ===\n")
print(prompt)
print("\n=== Model Responses ===\n")
print(responses.text)

```

To verify your work for Task 1.6, click **Check my progress** in the lab instructions.

Task 2. Retrieving and integrating knowledge with multimodal retrieval augmented generation (RAG)

To complete Task 2, follow the instructions at the top of each notebook cell:

- Run the cells with the comment "RUN THIS CELL AS IS".

- Complete and run the cells with the comment "COMPLETE THE MISSING PART AND RUN THIS CELL".

For additional information about the available data and helper functions for Task 2, review the section named **Available data and helper functions for Task 2** in the lab instructions.

Setup and requirements for Task 2

Import libraries

```
In [ ]: # "RUN THIS CELL AS IS"

# Import necessary packages and libraries.

from IPython.display import Markdown, display
from google.genai import types
from google.genai.types import Part
from PIL import Image as PIL_Image
```

Load the Gemini 3.5 Flash model

```
In [ ]: # "RUN THIS CELL AS IS"

# Load the Gemini 3.5 Flash model.

multimodal_model = "gemini-3.5-flash"
```

Download custom Python modules and utilities

The cell below will download some helper functions needed for this notebook, to improve readability. You can also view the code (`intro_multimodal_rag_utils.py`) directly on [GitHub](#).

```
In [ ]: # "RUN THIS CELL AS IS"

# Import necessary packages and libraries.
import os
import sys

if not os.path.exists("utils"):
    os.makedirs("utils")

# The utility file is pre-installed in this lab environment.
# No need to download it.
```

Get documents and images from Cloud Storage

```
In [ ]: # "RUN THIS CELL AS IS"

# Download documents and images used in this notebook.
```

```
!gcloud storage rsync -r gs://spl/s/gsp520 .
print("Download completed")
```

Task 2.1. Build metadata of documents containing text and images

Note: These steps may take a few minutes to complete.

Import helper functions to build metadata

```
In [ ]: # "RUN THIS CELL AS IS"

# Import helper functions from utils.
from utils.intro_multimodal_rag_utils import get_document_metadata
```

Explore the variables of the task

```
In [ ]: # "RUN THIS CELL AS IS"

# You're going to work with provided variables in this task.
# First, review and describe the content/purpose of each variable below.

# Specify the "PDF folder path" with single PDF and "PDF folder" with multiple PDF.

pdf_folder_path = "Google_Branding/" # if running in Agent Platform Workbench.

# Specify the image description prompt. Change it
image_description_prompt = """Explain what is going on in the image.
If it's a table, extract all elements of the table.
If it's a graph, explain the findings in the graph.
Do not include any numbers that are not mentioned in the image.
"""
```

Extract and store metadata of text and images from a document

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Call the "get_document_metadata" function from the utils file to extract text and
# text_metadata_df: This will contain extracted text snippets, their corresponding
# image_metadata_df: This will contain descriptions of the images found in the PDF

text_metadata_df, image_metadata_df = get_document_metadata(
    multimodal_model,
    pdf_folder_path,
    image_save_dir="images",
    image_description_prompt=image_description_prompt,
    embedding_size=1408,
)
```

```
print("\n\n --- Completed processing. ---")
```

Inspect the processed text metadata

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Explore the text_metadata_df dataframe by displaying the first few rows of the da
text_metadata_df.head()
```

Import the helper functions to implement RAG

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Import helper functions from utils.

from utils.intro_multimodal_rag_utils import (
    get_similar_text_from_query,
    print_text_to_text_citation,
    get_similar_image_from_query,
    print_text_to_image_citation,
    get_gemini_response,
    display_images,
)
```

Proceed to Task 2.2 below (no progress check for Task 2.1 in lab instructions).

Task 2.2. Create a user query

Explore the variables of the task

```
In [ ]: # "RUN THIS CELL AS IS"

# You're going to work with provided variables in this task.
# First, review and describe the content/purpose of each variable below.

query = """Questions:
- What are the key expectations that users can have from Google regarding the prov
- What specific rules and guidelines are established for users when using Google se
- How does Google handle intellectual property rights related to the content found
- What legal rights and remedies are available to users in case of problems or disa
- How do the service-specific additional terms interact with these general Terms of
"""
```

Proceed to Task 2.3 below (no progress check for Task 2.2 in lab instructions).

Task 2.3. Get all relevant text chunks

Retrieve relevant chunks of text based on the query

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Call the "get_similar_text_from_query" function from the utils file to retrieve r
# matching_results_chunks_data: This dictionary will contain file_name, page_num, c

matching_results_chunks_data = get_similar_text_from_query(
    query,
    text_metadata_df,
    column_name="text_embedding_chunk",
    top_n=10,
    chunk_text=True,
)
```

Display the first item of the text chunk dictionary

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Explore the first item in your matching_results_chunks_data dictionary by display

print_text_to_text_citation(
    matching_results_chunks_data,
    print_top=False,
    chunk_text=True,
)
```

Proceed to Task 2.4 below (no progress check for Task 2.3 in lab instructions).

Task 2.4. Create context text

Create a list to store the combined chunks of text

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Create an empty list named "context_text". This list will be used to store the co
context_text = list()
```

Iterate through each item in the text chunks dictionary

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Create a for loop to iterate through each item in the matching_results_chunks_dat

for key, value in matching_results_chunks_data.items():
    context_text.append(value["chunk_text"])
```

Join all the text chunks and store in a list

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Take all of the individual text chunks stored in the context_text list and join t
```

```
final_context_text = "\n".join(context_text)
```

Proceed to Task 2.5 below (no progress check for Task 2.4 in lab instructions).

Task 2.5. Pass context to Gemini

Explore the variables of the task

```
In [ ]: # "RUN THIS CELL AS IS"

# You're going to work with provided variables in this task. First, review and desc

prompt = f""" Instructions: Compare the images and the text provided as Context: to
Make sure to think thoroughly before answering the question and put the necessary s
If unsure, respond, "Not enough context to answer".

Context:
- Text Context:
{final_context_text}

{query}

Answer:
"""
```

Generate Gemini response with streaming output

```
In [ ]: # "COMPLETE THE MISSING PART AND RUN THIS CELL"

# Call "get_gemini_response" function from utils module in order to generate Gemini
# Format the streamed output using Markdown syntax for easy readability and convers

Markdown(
    get_gemini_response(
        multimodal_model,
        model_input=[prompt],
        stream=True,
        generation_config=GenerationConfig(temperature=0.2,max_output_tokens=2048),
    )
)

# HERE WE WLL GET ERROR BUT OUR LAB WILL BE COMPLETED SO DON'T WORRY ABOUT THIS ERR
```

To verify your work for Task 2.5, click **Check my progress** in the lab instructions.